Terminal-Bench 3 & 4

Manager-Ready Research Guide + TB3 Task Acceptance Matrix

Verified snapshot: 2 September 2026

Purpose. This document is designed to answer the project requirement: study Terminal-Bench 3 for the task-acceptance/implementation matrix, then study Terminal-Bench 4 for the differences and quality changes.

Bottom line. For TB3, the current public *task-implementation* rubric contains **28 named criteria**. TB4 is primarily a maintenance/quality release of the TB3 task set, with task removals/revisions and resource/timeout calibration.

Important scope clarification. The number 28 refers specifically to the current public TB3 *task-implementation rubric*. It should not be described as the total number of every automated or human acceptance check. The current review pipeline includes additional checks around the implementation rubric. Some current secondary documentation describes a broader review process using “35 rubric criteria”; that is a broader scope/terminology description, not a replacement for the 28 criteria in the public task-implementation.toml.

1. What Terminal-Bench Measures

Terminal-Bench evaluates AI agents on complex terminal/computer-use tasks. The harness runs agents in controlled environments, records their interaction, and verifies the resulting state. The core benchmark outcome is **resolution rate** (whether the task is successfully resolved under the benchmark’s verification).

For the project, the key idea is: **a good benchmark task is not merely a difficult question.** It should resemble meaningful professional computer work, require multi-step agent interaction, have a robust and reproducible environment, and have verification that can reliably distinguish success from failure.

2. TB3: What Researchers Want From a Task

Verifiable outcome: The final state should be objectively and efficiently testable.

Real difficulty: The hard part should require professional reasoning/work, not arbitrary formatting or obscure facts.

Agentic interaction: The task should require meaningful multi-step terminal actions.

Robustness: Tests should resist cheating/reward hacking and should test actual behavior.

Reproducibility: Runs should be stable; dependencies/data should be pinned where appropriate.

Instruction↔test alignment: The instructions and assertions should match exactly.

Reviewability: Another reviewer should be able to understand why the task is valid and how it is verified.

3. TB3 Task Acceptance / Implementation Matrix — 28 Criteria

Use this as the practical checklist for deciding whether a proposed task is ready for review. “Pass condition” is a concise interpretation of the current public criterion; for exact wording, use the source file listed in the sources section.

#	Criterion	What it requires	Practical check
1	**verifiable**	The task must have reliable, deterministic, efficient verification.	Use executable tests/verifiers; avoid fragile or unjustified LLM-only judging.
2	**solvable**	A working solution must be possible and pass all required tests; an expert should be able to implement it in a few focused hours.	Validate an oracle/solution before submission.
3	**difficult**	The task should require professional expertise and be meaningfully beyond routine undergraduate work.	Difficulty should come from reasoning/work, not tedium, obscure facts, or tricks.
4	**interesting**	The task should represent useful real-world professional work.	Prefer meaningful engineering/data/ops/research workflows.
5	**outcome_verified**	Verification should focus on the final outcome/state, not merely the sequence of actions.	An agent can use a different valid approach and still pass.
6	**anti_cheat_robustness**	Tests should resist fake wrappers, monkey-patching, answer leaks, modifying tests, caching, or similar reward-hacking strategies.	Try adversarial/cheating paths during review.
7	**task_security**	The task must not introduce malicious behavior or unsafe mechanisms such as credential/network exfiltration, supply-chain attacks, host escape, destructive operations, obfuscated code, prompt injection, or cross-task interference.	Review all task files and setup paths for security risks.
8	**functional_verification**	Verification should execute relevant behavior rather than checking superficial source-code keywords.	Test what the user was actually asked to produce.
9	**deterministic_reproducible**	The task should behave consistently across runs; task-relevant mutable live services should be avoided or pinned.	Pin dependencies/data where appropriate.
10	**essential_difficulty**	The core challenge should require genuine reasoning/work rather than formatting minutiae or arbitrary constraints.	Remove difficulty that does not contribute to the intended task.
11	**test_instruction_alignment**	Every important test assertion should map to the instructions, and the instructions should not require untested outcomes.	Check the instruction↔test contract both ways.
12	**novel**	The task should not be a textbook exercise or primarily a memorization task.	Prefer realistic, non-trivial scenarios.
13	**agentic**	The task should require multi-step terminal interaction and meaningful agent actions.	Avoid tasks solvable by a single trivial command.
14	**reviewable**	A reviewer who is not the original author should be able to understand and assess the task.	Provide enough explanation/context for review.
15	**instruction_concision**	Instructions should be human-written, concise, unambiguous, and avoid giving a step-by-step solution recipe.	Use absolute paths where needed; avoid solution hints.
16	**solution_quality**	The reference solution should perform genuine computation/work rather than hard-coding the expected answer.	Keep the oracle general enough to demonstrate a real solution.
17	**environment_hygiene**	Tests/solution should not be exposed in the agent runtime image, and test-only dependencies should not be unnecessarily baked into the environment; clean up package-manager state as appropriate.	Separate agent environment from verification assets.
18	**structured_data_schema**	If structured output is required, its normative schema must be precise and testable.	Define required fields/types/constraints clearly.
19	**typos**	Critical identifiers, commands, paths, filenames, and other important text should be free of errors.	A typo in a path or command can invalidate an otherwise good task.
20	**difficulty_explanation_quality**	The explanation should state the intrinsic challenge, include realistic data context where applicable, and identify the real-world role/use case.	Explain why the task is hard—not merely that it is hard.
21	**solution_explanation_quality**	The solution explanation should describe the high-level strategy and remain consistent with the submitted solution.	Do not provide an unrelated or misleading explanation.
22	**verification_explanation_quality**	The verification explanation should state concretely how correctness is checked and justify any tolerance/range used.	Make pass/fail logic understandable.
23	**category_and_tags**	Category and tags should be meaningful and specific to the task.	Avoid generic or misleading labels.
24	**task_name**	The task name should follow the current public TB3 rubric convention: lowercase kebab-case and no more than 5 hyphen-separated words.	Keep it concise and descriptive.
25	**resource_configuration**	Time, CPU, memory, storage, and GPU resources should be appropriate; difficulty should be hard for the agent, not unnecessarily hard for the computer.	Calibrate resources using actual runs.

#	Criterion	What it requires	Practical check
26	**task_readme**	An optional README can provide useful reviewer context.	Use it when reviewer-facing context would materially help.
27	**expert_time_estimate**	The expert time estimate should be non-zero, plausible, and represent best-case focused work by a domain expert.	Use a realistic estimate rather than a placeholder.
28	**task_toml_schema**	The task metadata should use supported Harbor/TB3 fields and conform to the expected schema.	Run schema/static checks before submission.

4. TB3 Acceptance: High-Level Workflow

Stage	Action	What to prove
1	**Propose**	Write a concise task that represents meaningful professional work.
2	**Build**	Create instruction.md, Docker environment, solution/oracle, tests, and task metadata.
3	**Validate**	Run static checks, schema/check tooling, build, oracle and nop validation.
4	**Trial with agents**	Run actual agents and inspect trajectories; ensure the task is genuinely difficult and not broken.
5	**Security/anti-cheat review**	Check for prompt injection, exfiltration, test leakage, reward hacking, and other unsafe or gameable behavior.
6	**Review & iterate**	Fix issues found by automated/human review and resubmit.

5. TB4 — What Changed From TB3

TB4 should be understood mainly as a **continuous quality/maintenance release** of TB3 rather than a completely new benchmark family. The task set was cleaned up, tasks were revised, and compute/time budgets were calibrated.

Area	TB3	TB4	Why it matters
Task-set purpose	TB3 was the major task release/contribution cycle.	TB4 is primarily a maintenance/quality release of TB3.	Use TB3 for rubric/task-design study; use TB4 to understand quality-control changes.
Task count	74 tasks in the shipped TB3 v0.1 release; the earlier contribution call targeted about 100 tasks.	66 tasks in TB4 v4.0.0.	Do not confuse the TB3 contribution target (~100) with the shipped count (74).
Removed tasks	—	8 tasks removed.	Removal was part of quality/saturation/refusal/public-solution/platform cleanup.
Revised tasks	Baseline task set.	20 tasks revised/fixed according to current Snorkel TB4 documentation.	Revisions included task fixes and resource/timeout calibration.
New tasks in TB4	—	0 new tasks added in the TB4 maintenance cycle.	TB4 should not be described as a fresh task-generation release.
Timeout/resources	TB3 had task-specific resource configuration; template defaults are configurable.	TB4 increased/standardized execution budgets and calibrated CPU/memory for some tasks.	Some prior failures were attributable to insufficient resource budgets rather than agent inability.
Evaluation metric	Resolution/pass rate is the core benchmark metric.	Resolution rate remains the core metric, now over the revised 66-task set.	TB3 and TB4 percentages are not directly apples-to-apples because the task set/conditions changed.
Leaderboard interpretation	TB3 leaderboard measures agent success on the TB3 task set.	TB4 leaderboard measures success on the revised TB4 task set.	Always report version, task count, harness/agent, and trial methodology with a score.

6. How to Read TB3 vs TB4 Scores

Core metric: resolution rate. A higher percentage means the agent successfully resolved more benchmark tasks under the specified evaluation setup.

Do not compare a TB3 percentage to a TB4 percentage as if they were the same experiment. TB4 uses a revised 66-task set and changed execution/resource conditions. A score must be reported with its benchmark version, task set, agent/harness configuration, and trial methodology.

Current TB4 leaderboard snapshot (for context, not a permanent ranking):

Agent/model setup	Resolution rate
Opus 5 + Claude Code	51.8% ± 3.4
Fable 5 + Claude Code	44.5% ± 3.8
GLM-5.3 + Claude Code	41.8% ± 3.2
GPT-5.6 Sol + Codex	37.3% ± 3.8
Opus 4.8	23.6%
GPT-5.6 Terra	21.5%
Grok 4.6	20.3%
GPT-5.6 Luna	17.3%
Grok 4.5	12.4%
Sonnet 5	12.4%

The leaderboard is dynamic. Treat the values above as a dated snapshot rather than fixed benchmark facts.

7. What “Good” Looks Like for Our Project

For TB3: the strongest deliverable is the 28-row acceptance matrix plus a clear explanation of each criterion and how a reviewer can test it.

For TB4: the strongest deliverable is a difference table showing task-set cleanup, revisions/removals, resource/timeout calibration, and the resulting comparability caveat.

For management: the key message is that benchmark quality depends on both *task design* and *verification integrity*. A task can be difficult but still be rejected if it is ambiguous, non-reproducible, insecure, gameable, poorly verified, or misconfigured.

8. Manager Checklist — Ready for Submission?

■ The task represents meaningful professional work.

■ The task requires multi-step terminal/agent interaction.

■ The instructions are concise and do not reveal a solution recipe.

■ Every important instruction has corresponding verification.

■ Verification checks actual behavior/outcome.

■ The reference solution is genuine, not hard-coded.

■ The task is deterministic/reproducible.

■ Security and anti-cheat risks have been reviewed.

■ Tests/solution are not leaked into the agent runtime image.

■ Resources are sufficient but not artificially generous.

■ Metadata, task name, schema, explanations, tags, and time estimate are complete.

■ Actual agent trials show the task is challenging without being broken.

9. Source & Verification Notes

The following sources were used for this version. URLs are written plainly so the PDF remains useful outside ChatGPT. Verified against the current public pages/files on 2 September 2026.

Source	URL	Supports
Terminal-Bench — TB3 contribution call	https://www.tbench.ai/news/tb3-contribution-call	TB3 task-design goals, good-task characteristics, contribution workflow.
Terminal-Bench — benchmark announcement	https://www.tbench.ai/news/announcement	Benchmark purpose, harness/verification concepts, Terminus context.
Terminal-Bench — benchmarks	https://www.tbench.ai/benchmarks	Current benchmark/version overview.
Terminal-Bench — leaderboard	https://www.tbench.ai/leaderboard	Current leaderboard/version context; resolution rate.
Terminal-Bench 3 public — task implementation rubric	https://raw.githubusercontent.com/scaleapi/terminal-bench-3-public/main/rubrics/task-implementation.toml	Current public 28-criterion task-implementation rubric.
Terminal-Bench 3 public — review automation	https://github.com/scaleapi/terminal-bench-3-public/blob/main/TASK_REVIEW_AUTOMATION.md	Broader automated review pipeline around the implementation rubric.
Terminal-Bench 3 public — contributing guide	https://github.com/scaleapi/terminal-bench-3-public/blob/main/CONTRIBUTING.md	Task structure, local validation, submission workflow.
Terminal-Bench 3 public — task template	https://github.com/scaleapi/terminal-bench-3-public/blob/main/task-template.toml	Task metadata/resource template fields.
Terminal-Bench — leaderboard integrity update	https://www.tbench.ai/news/leaderboard-integrity-update	Trajectory/integrity and reward-hacking policy.
Terminal-Bench — Challenges	https://www.tbench.ai/news/terminal-bench-challenges	Separate long-horizon/token-intensive benchmark format.
Snorkel — Terminal-Bench 4.0	https://snorkel.ai/leaderboard/terminal-bench-4-0/	TB4 task count, removals, revisions, current leaderboard snapshot/methodology.
Snorkel — Terminal-Bench 4 continuous QA	https://snorkel.ai/blog/terminal-bench-4-continuous-qa/	TB4 maintenance, task fixes, resource/timeout calibration.

Terminology note: The public TB3 task-implementation TOML currently contains 28 named criteria. The broader review process includes additional automated and human checks. Therefore, in a manager presentation, say “28 current task-implementation rubric criteria” rather than simply “28 total acceptance checks.”

Also: TB3’s original contribution call targeted approximately 100 diverse tasks, while the shipped TB3 v0.1 release contained 74 tasks. These are different figures and should not be presented as a contradiction.

Scope: This report focuses on the project’s requested TB3 matrix and TB4 differences. Terminal-Bench Challenges is a separate benchmark format and should not be mixed into the core TB3/TB4 task-acceptance matrix.